C++ para Programação Competitiva: STL, Heaps e Iteradores

1 Priority Queue e Heaps

1.1 Conceito de Heap

Um heap é uma estrutura que mantém o elemento extremo acessível:

- Max-heap: maior elemento no topo
- Min-heap: menor elemento no topo

Complexidade:

$$\text{push} = O(\log n), \quad \text{pop} = O(\log n), \quad \text{top} = O(1)$$

1.2 Priority Queue padrão (max-heap)

```
priority_queue<int> pq;
pq.push(3);
pq.push(10);
pq.push(5);

cout << pq.top(); // 10
pq.pop();
```

1.3 Min-heap

```
priority_queue<int, vector<int>, greater<int>> pq;
```

1.4 Comparator customizado

```
auto cmp = [](pair<int,int> a, pair<int,int> b){
    return a.second > b.second;
};

priority_queue<pair<int,int>, vector<pair<int,int>>, decltype(cmp)> pq(cmp
    );
```

1.5 Casos clássicos

- Dijkstra (min-heap)
- Top-K elementos
- Merge de listas ordenadas
- Scheduling (greedy)

1.6 Limitações

- Não permite remover elementos arbitrários
- Não permite acesso interno

2 Heaps manuais

```
vector<int> v = {3,1,5};

make_heap(v.begin(), v.end());
pop_heap(v.begin(), v.end());
v.pop_back();

v.push_back(10);
push_heap(v.begin(), v.end());
```

Min-heap:

```
make_heap(v.begin(), v.end(), greater<int>());
```

3 Iteradores

3.1 Definição

Iteradores são ponteiros generalizados que percorrem containers.

```
vector<int> v = {1,2,3};

for(auto it = v.begin(); it != v.end(); it++){
    cout << *it;
}
```

3.2 Tipos de iteradores

- Random access: vector, deque
- Bidirectional: set, map
- Forward: unordered_map

3.3 Operações importantes

```
auto it = v.begin();
advance(it, k);
distance(a, b);
```

3.4 Iteradores reversos

```
for(auto it = v.rbegin(); it != v.rend(); it++){
}
```

3.5 Segurança

Forma incorreta:

```
v.erase(it); // invalida iterador
```

Forma correta:

```
it = v.erase(it);
```

4 Iteradores em set/map

```
set<int> s = {1,3,5};

auto it = s.lower_bound(3);
auto it2 = s.upper_bound(3);
```

5 Comparação: priority_queue vs set

Estrutura	Inserção	Remoção arbitrária	Ordenação
priority_queue	$O(\log n)$	Não	Parcial
set	$O(\log n)$	Sim	Total

6 Padrões com iteradores

6.1 Range-based loop

```
for(auto x : v)
```

6.2 Loop com índice

```
for(int i = 0; i < v.size(); i++)
```

7 Dicas avançadas

7.1 Lazy deletion em heap

```
unordered_map<int,int> removed;
```

7.2 Heap com struct

```
struct Node{
    int dist, v;
    bool operator<(const Node& other) const{
        return dist > other.dist;
    }
};

priority_queue<Node> pq;
```

8 Complexidade

Estrutura	Inserção	Remoção	Busca
vector	$O(1)$*	$O(n)$	$O(n)$
set/map	$O(\log n)$	$O(\log n)$	$O(\log n)$
priority_queue	$O(\log n)$	$O(\log n)$	topo $O(1)$

9 Exemplo 1: Dijkstra (priority_queue)

Problema: encontrar menor caminho em grafo ponderado.

```
const int INF = 1e9;
vector<vector<pair<int,int>>> adj; // {vizinho, peso}

vector<int> dijkstra(int n, int src){
    vector<int> dist(n, INF);
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;

    dist[src] = 0;
    pq.push({0, src});

    while(!pq.empty()){
        auto [d, u] = pq.top();
        pq.pop();

        if(d > dist[u]) continue;

        for(auto [v, w] : adj[u]){
            if(dist[v] > d + w){
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
```

```
        }
    }
    return dist;
}
```

Ideia: sempre expandir o nó com menor distância atual.

10 Exemplo 2: Top-K elementos

Problema: manter os K maiores elementos.

```
priority_queue<int, vector<int>, greater<int>> pq;

for(int x : v){
    pq.push(x);
    if(pq.size() > k) pq.pop();
}
```

Ideia: min-heap guarda apenas os K maiores.

11 Exemplo 3: Sweep Line com multiset

Problema: máximo número de intervalos ativos.

```
vector<pair<int,int>> events;

for(auto [l, r] : intervals){
    events.push_back({l, 1});
    events.push_back({r, -1});
}

sort(events.begin(), events.end());

int cur = 0, ans = 0;
for(auto [x, type] : events){
    cur += type;
    ans = max(ans, cur);
}
```

Ideia: transformar intervalos em eventos.

12 Exemplo 4: Sliding Window (two pointers)

Problema: maior subarray com soma $\leq S$.

```
int l = 0, sum = 0, ans = 0;

for(int r = 0; r < n; r++){
    sum += v[r];
    while(sum > S){
        sum -= v[l++];
    }
    ans = max(ans, r - l + 1);
```

```
}
```

13 Exemplo 5: Uso de set (ordenado)

Problema: encontrar menor elemento $\geq x$.

```
set<int> s = {1,3,5,7};

auto it = s.lower_bound(4);
if(it != s.end()){
    cout << *it; // 5
}
```

Ideia: busca binária em estrutura dinâmica.

14 Exemplo 6: Merge de K listas

```
priority_queue<
    pair<int,int>,
    vector<pair<int,int>>,
    greater<>
> pq;
```

Ideia: sempre pegar o menor entre os heads.

15 Exemplo 7: Frequência com map

```
map<int,int> freq;

for(int x : v){
    freq[x]++;
}
```

16 Exemplo 8: Binary Search

```
int l = 0, r = n-1;

while(l <= r){
    int m = (l + r) / 2;
    if(v[m] == x) return m;
    else if(v[m] < x) l = m + 1;
    else r = m - 1;
}
```

17 Exemplo 9: Prefix Sum

```
vector<int> pref(n+1, 0);

for(int i = 0; i < n; i++){
    pref[i+1] = pref[i] + v[i];
}

// soma intervalo [l, r]
int sum = pref[r+1] - pref[l];
```

18 Exemplo 10: DFS

```
vector<vector<int>> adj;
vector<bool> vis;

void dfs(int u){
    vis[u] = true;
    for(int v : adj[u]){
        if(!vis[v]) dfs(v);
    }
}
```

19 Conclusão

Esses padrões cobrem grande parte dos problemas de programação competitiva:

- Grafos: Dijkstra, DFS/BFS
- Estruturas: heap, set, map
- Técnicas: two pointers, sweep line, prefix sum
- Busca: binary search

20 Busca Binária e Funções Associadas (STL)

20.1 Pré-requisito

Todas essas funções assumem que o container está ordenado:

```
sort(v.begin(), v.end());
```

20.2 binary_search

Verifica se um elemento existe.

```
bool exists = binary_search(v.begin(), v.end(), x);
```

Retorna true se $x \in v$, caso contrário false

Complexidade: $O(\log n)$

20.3 lower_bound

Retorna um iterador para o **primeiro elemento** $\geq x$.

```
auto it = lower_bound(v.begin(), v.end(), x);
```

Interpretação:

$$\text{it} = \min\{i \mid v[i] \geq x\}$$

Exemplo:

```
v = {1,2,4,4,5};

lower_bound(v.begin(), v.end(), 4) -> aponta para o primeiro 4
lower_bound(v.begin(), v.end(), 3) -> aponta para 4
```

20.4 upper_bound

Retorna um iterador para o **primeiro elemento** $> x$.

```
auto it = upper_bound(v.begin(), v.end(), x);
```

Interpretação:

$$\text{it} = \min\{i \mid v[i] > x\}$$

Exemplo:

```
v = {1,2,4,4,5};

upper_bound(v.begin(), v.end(), 4) -> aponta para 5
```

20.5 Contagem de ocorrências

Número de vezes que x aparece:

```
int count = upper_bound(v.begin(), v.end(), x)
          - lower_bound(v.begin(), v.end(), x);
```

$$\text{count} = |\{i : v[i] = x\}|$$

20.6 Busca binária manual

```
int l = 0, r = n-1;

while(l <= r){
    int m = l + (r - l) / 2;
    if(v[m] == x) return m;
    else if(v[m] < x) l = m + 1;
    else r = m - 1;
}
```

20.7 Binary search em resposta (muito importante)

Usado quando a resposta é monotônica.

```
int l = 0, r = 1e9;

while(l < r){
    int m = (l + r) / 2;
    if(check(m)) r = m;
    else l = m + 1;
}
```

Objetivo: encontrar o menor x tal que $check(x) = true$

20.8 Uso em set/map

```
set<int> s = {1,3,5,7};

auto it = s.lower_bound(4); // aponta para 5
auto it2 = s.upper_bound(5); // aponta para 7
```

Complexidade: $O(\log n)$

20.9 Resumo conceitual

Função	Significado
binary_search	verifica existência
lower_bound	primeiro $\geq x$
upper_bound	primeiro $> x$

20.10 Dicas práticas

- Sempre garantir que o vetor está ordenado
- Usar lower_bound para encontrar posição de inserção
- Diferença de iteradores dá índice:

```
int idx = it - v.begin();
```

- Evitar overflow:

$$m = l + \frac{r - l}{2}$$

21 Técnicas Avançadas de Busca e Estruturas

21.1 Ternary Search

Usado para funções unimodais (cresce e depois decresce, ou vice-versa).

Caso contínuo:

```
double ternary(double l, double r){
    for(int i = 0; i < 100; i++){
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;

        if(f(m1) < f(m2)) l = m1;
        else r = m2;
    }
    return l;
}
```

Caso discreto:

```
int ternary(int l, int r){
    while(r - l > 3){
        int m1 = l + (r - l) / 3;
        int m2 = r - (r - l) / 3;

        if(f(m1) < f(m2)) l = m1;
        else r = m2;
    }

    int ans = l;
    for(int i = l; i <= r; i++){
        if(f(i) < f(ans)) ans = i;
    }
    return ans;
}
```

21.2 Busca binária em double

Usada quando a resposta é contínua.

```
double l = 0, r = 1e9;

for(int i = 0; i < 100; i++){
    double m = (l + r) / 2;
    if(check(m)) r = m;
    else l = m;
}
```

Critério alternativo:

$$r - l < \varepsilon$$

21.3 Monotonic Stack

Mantém elementos em ordem monotônica.

Exemplo: próximo elemento maior à direita

```
vector<int> next_greater(n, -1);
stack<int> st;

for(int i = 0; i < n; i++){
    while(!st.empty() && v[st.top()] < v[i]){
        next_greater[st.top()] = v[i];
        st.pop();
    }
    st.push(i);
}
```

Complexidade: $O(n)$

21.4 Monotonic Deque (Sliding Window)

Usado para máximo/mínimo em janela.

```
deque<int> dq;

for(int i = 0; i < n; i++){
    while(!dq.empty() && dq.front() <= i - k)
        dq.pop_front();

    while(!dq.empty() && v[dq.back()] <= v[i])
        dq.pop_back();

    dq.push_back(i);

    if(i >= k-1)
        cout << v[dq.front()] << "␣";
}
```

21.5 Resumo conceitual

Técnica	Uso
Binary Search	função monotônica
Ternary Search	função unimodal
Double Search	precisão contínua
Monotonic Stack	nearest greater/smaller
Monotonic Deque	sliding window max/min

21.6 Dicas práticas

- Binary search: condição monotônica clara
- Ternary search: evitar se função não for unimodal
- Double search: usar número fixo de iterações
- Monotonic stack: cada elemento entra e sai no máximo uma vez
- Deque: mantém candidatos relevantes apenas

22 Árvores, DFS e BFS

22.1 Definição de Árvore

Uma árvore é um grafo não direcionado, conexo e sem ciclos.

Propriedades:

- Possui n vértices e $n-1$ arestas
- Existe exatamente um caminho entre quaisquer dois nós
- Pode ser enraizada (rooted tree)

Representação comum:

```
vector<vector<int>> adj(n);
```

22.2 DFS (Depth-First Search)

Explora o grafo em profundidade.

22.2.1 Implementação recursiva

```
vector<vector<int>> adj;
vector<bool> vis;

void dfs(int u){
    vis[u] = true;
    for(int v : adj[u]){
        if(!vis[v]){
```

```
            dfs(v);
        }
    }
}
```

22.2.2 Complexidade

$$O(n+m)$$

22.2.3 Usos clássicos

- Encontrar componentes conexas
- Detectar ciclos
- Ordenação topológica
- DP em árvores

22.3 BFS (Breadth-First Search)

Explora o grafo em camadas (níveis).

22.3.1 Implementação

```
vector<vector<int>> adj;
vector<int> dist(n, -1);

void bfs(int src){
    queue<int> q;
    q.push(src);
    dist[src] = 0;

    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : adj[u]){
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

22.3.2 Complexidade

$$O(n+m)$$

22.3.3 Usos clássicos

- Menor caminho em grafos não ponderados
- Verificação de bipartição
- Expansão por níveis

22.4 Diferença entre DFS e BFS

Característica	DFS	BFS
Estrutura	Stack / Recursão	Queue
Ordem	Profundidade	Níveis
Menor caminho	Não garante	Garante (não ponderado)

22.5 DFS em árvore (com pai)

```
void dfs(int u, int p){
    for(int v : adj[u]){
        if(v != p){
            dfs(v, u);
        }
    }
}
```

22.6 Altura e profundidade

- Profundidade: distância da raiz até o nó
- Altura: maior distância até uma folha

22.7 Exemplo: calcular profundidade

```
vector<int> depth;

void dfs(int u, int p){
    for(int v : adj[u]){
        if(v != p){
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }
    }
}
```

22.8 Observações práticas

- Sempre marcar nós visitados para evitar loops
- Em árvores, usar parâmetro `parent` ao invés de `visited`
- BFS usa mais memória que DFS

22.9 Lista de Adjacência (adj)

A lista de adjacência é a forma padrão de representar grafos.

22.9.1 Definição

```
vector<vector<int>> adj(n);
```

Interpretação:

$$adj[u] = \{\text{vizinhos de } u\}$$

22.9.2 Construção

Grafo não direcionado:

```
adj[u].push_back(v);
adj[v].push_back(u);
```

Grafo direcionado:

```
adj[u].push_back(v);
```

22.9.3 Grafo com pesos

```
vector<vector<pair<int,int>>> adj(n);
```

$$adj[u] = \{(v, w)\}$$

22.9.4 Iteração

```
for(int v : adj[u]) {
    // processa vizinho
}
```

Com pesos:

```
for(auto [v, w] : adj[u]) {
    // usa v e w
}
```

22.9.5 Complexidade

- Memória: $O(n + m)$
- Iteração DFS/BFS: $O(n + m)$

22.9.6 Comparação com matriz

Estrutura	Memória	Iteração
Matriz	$O(n^2)$	Lenta
Lista (adj)	$O(n + m)$	Rápida

23 Problemas Clássicos com Lista de Adjacência (adj)

23.1 1. Componentes Conexas (DFS)

Problema: contar quantos componentes existem em um grafo.

Ideia: rodar DFS a partir de cada nó não visitado.

```
vector<vector<int>> adj;
vector<bool> vis;

void dfs(int u){
    vis[u] = true;
    for(int v : adj[u]){
        if(!vis[v]) dfs(v);
    }
}

int count_components(int n){
    vis.assign(n, false);
    int count = 0;

    for(int i = 0; i < n; i++){
        if(!vis[i]){
            dfs(i);
            count++;
        }
    }
    return count;
}
```

—

23.2 2. Menor Caminho (BFS)

Problema: menor distância em grafo não ponderado.

```
vector<int> dist(n, -1);

void bfs(int src){
    queue<int> q;
    q.push(src);
    dist[src] = 0;

    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : adj[u]){
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}
```

—

23.3 3. Detectar Ciclo (DFS)

Problema: verificar se um grafo não direcionado tem ciclo.

```
bool has_cycle(int u, int parent){
    vis[u] = true;
    for(int v : adj[u]){
        if(!vis[v]){
            if(has_cycle(v, u)) return true;
        } else if(v != parent){
            return true;
        }
    }
    return false;
}
```

—

23.4 4. Árvore: calcular profundidade

Problema: depth de cada nó.

```
vector<int> depth;

void dfs(int u, int p){
    for(int v : adj[u]){
        if(v != p){
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }
    }
}
```

—

23.5 5. Diâmetro de Árvore

Problema: maior distância entre dois nós.

Ideia: BFS duas vezes.

```
int bfs_far(int src){
    vector<int> dist(n, -1);
    queue<int> q;

    q.push(src);
    dist[src] = 0;

    int last = src;

    while(!q.empty()){
        int u = q.front(); q.pop();
        last = u;
        for(int v : adj[u]){
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
```

```
                q.push(v);
            }
        }
    }
    return last;
}
```

—

23.6 6. Bipartição (BFS)

Problema: verificar se o grafo é bipartido.

```
vector<int> color(n, -1);

bool is_bipartite(int src){
    queue<int> q;
    q.push(src);
    color[src] = 0;

    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int v : adj[u]){
            if(color[v] == -1){
                color[v] = color[u] ^ 1;
                q.push(v);
            } else if(color[v] == color[u]){
                return false;
            }
        }
    }
    return true;
}
```

—

23.7 7. Ordenação Topológica (DAG)

Problema: ordenar grafo direcionado acíclico.

```
vector<int> indeg(n, 0);

for(int u = 0; u < n; u++){
    for(int v : adj[u]){
        indeg[v]++;
    }
}

queue<int> q;
for(int i = 0; i < n; i++){
    if(indeg[i] == 0) q.push(i);
}

vector<int> topo;
```

```
while(!q.empty()){
    int u = q.front(); q.pop();
    topo.push_back(u);

    for(int v : adj[u]){
        if(--indeg[v] == 0){
            q.push(v);
        }
    }
}
```

—

23.8 Resumo de padrões

Problema	Técnica
Componentes	DFS
Menor caminho	BFS
Ciclo	DFS
Árvore	DFS com pai
Diâmetro	BFS x2
Bipartição	BFS + cores
Topológico	BFS (Kahn)

Algorithm	Main idea	Best use case	Data structure	Complexity
Boruvka	Each connected component chooses its minimum outgoing edge; all safe edges are added simultaneously.	Minimum spanning tree when parallelization is useful or when we want to merge many components at once.	Connected components, labels, Union-Find possible.	$O(E \log V)$
Prim-Jarnik	Grow one tree from an initial vertex; repeatedly add the cheapest edge leaving the current tree.	Minimum spanning tree on connected weighted graphs, especially dense graphs or when using priority queues.	Priority queue; Fibonacci heap for optimal bound.	$O(E + V \log V)$ with Fibonacci heap
Kruskal	Sort edges by increasing weight; add an edge if it connects two different components.	Minimum spanning tree, especially for sparse graphs. Very simple and practical.	Union-Find / Disjoint Set Union.	$O(E \log V)$

Table 1: Summary of minimum spanning tree algorithms.

24 Essential Graph Algorithms in C++

24.1 Graph Representation

We use an adjacency list. For weighted graphs, each edge is stored as a pair `(neighbor, weight)`.

Algorithm	Graph type	Edge weights allowed	Main characteristic
BFS	Unweighted graph, or graph where every edge has the same weight.	All weights equal, usually 1.	Finds shortest paths by exploring vertices layer by layer from the source.
DAG shortest paths	Directed acyclic graph.	Can allow negative weights.	Processes vertices in topological order; works because there are no directed cycles.
Dijkstra	Directed or undirected weighted graph.	Requires nonnegative weights.	Greedy algorithm: repeatedly finalizes the vertex with smallest tentative distance.
Bellman-Ford	Directed or undirected weighted graph.	Allows negative weights.	Repeatedly relaxes all edges; can detect negative cycles.
Johnson	Directed weighted graph.	Allows negative weights, but no negative cycles.	Solves all-pairs shortest paths by reweighting edges, then running Dijkstra from every vertex.

Table 2: Summary of shortest path algorithms.

```
#include <bits/stdc++.h>
using namespace std;

const long long INF = 4e18;

struct Edge {
    int u, v;
    long long w;
};

using Graph = vector<vector<pair<int, long long>>>;
// graph[u] contains pairs (v, w), meaning edge u -> v with weight w.
```

24.2 Breadth-First Search for Unweighted Shortest Paths

BFS computes shortest paths when all edges have the same weight, usually weight 1.

```
vector<int> bfs_shortest_paths(const vector<vector<int>>& graph, int
   source) {
    int n = graph.size();

    vector<int> dist(n, -1);
    vector<int> parent(n, -1);
    queue<int> q;

    dist[source] = 0;
```

```
    q.push(source);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : graph[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }

    return dist;
}
```

Complexity: `O(V + E)`.

24.3 Shortest Paths in a DAG

For a directed acyclic graph, we process vertices in topological order and relax all outgoing edges.

```
void dfs_topological(
    int u,
    const Graph& graph,
    vector<bool>& visited,
    vector<int>& order
) {
    visited[u] = true;

    for (auto [v, w] : graph[u]) {
        if (!visited[v]) {
            dfs_topological(v, graph, visited, order);
        }
    }

    order.push_back(u);
}

vector<long long> dag_shortest_paths(const Graph& graph, int source) {
    int n = graph.size();

    vector<bool> visited(n, false);
    vector<int> order;

    for (int u = 0; u < n; u++) {
        if (!visited[u]) {
            dfs_topological(u, graph, visited, order);
        }
    }

    reverse(order.begin(), order.end());
```

```
    vector<long long> dist(n, INF);
    dist[source] = 0;

    for (int u : order) {
        if (dist[u] == INF) continue;

        for (auto [v, w] : graph[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    return dist;
}
```

Complexity: `O(V + E)`.

24.4 Dijkstra's Algorithm

Dijkstra computes shortest paths from one source when all edge weights are nonnegative.

```
vector<long long> dijkstra(const Graph& graph, int source) {
    int n = graph.size();

    vector<long long> dist(n, INF);
    dist[source] = 0;

    priority_queue<
        pair<long long, int>,
        vector<pair<long long, int>>,
        greater<pair<long long, int>>
    > pq;

    pq.push({0, source});

    while (!pq.empty()) {
        auto [du, u] = pq.top();
        pq.pop();

        if (du != dist[u]) continue;

        for (auto [v, w] : graph[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    return dist;
}
```

Complexity with binary heap: `O(E log V)`.

24.5 Bellman-Ford Algorithm

Bellman-Ford allows negative weights and detects negative cycles reachable from the source.

```
pair<vector<long long>, bool> bellman_ford(
    int n,
    const vector<Edge>& edges,
    int source
) {
    vector<long long> dist(n, INF);
    dist[source] = 0;

    for (int i = 0; i < n - 1; i++) {
        bool changed = false;

        for (const Edge& e : edges) {
            if (dist[e.u] == INF) continue;

            if (dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        }

        if (!changed) break;
    }

    bool has_negative_cycle = false;

    for (const Edge& e : edges) {
        if (dist[e.u] == INF) continue;

        if (dist[e.u] + e.w < dist[e.v]) {
            has_negative_cycle = true;
            break;
        }
    }

    return {dist, has_negative_cycle};
}
```

Complexity: `O(VE)`.

24.6 Union-Find / Disjoint Set Union

Union-Find is used by Kruskal's algorithm to test whether adding an edge creates a cycle.

```
struct DSU {
    vector<int> parent;
    vector<int> size;

    DSU(int n) {
        parent.resize(n);
        size.assign(n, 1);
```

```
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        if (parent[x] == x) return x;
        return parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);

        if (a == b) return false;

        if (size[a] < size[b]) {
            swap(a, b);
        }

        parent[b] = a;
        size[a] += size[b];

        return true;
    }
};
```

Amortized complexity per operation: almost constant.

24.7 Kruskal's Algorithm

Kruskal computes a minimum spanning tree by sorting edges and adding the cheapest edges that do not create cycles.

```
long long kruskal(int n, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return a.w < b.w;
    });

    DSU dsu(n);

    long long total_weight = 0;
    int edges_used = 0;

    for (const Edge& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            total_weight += e.w;
            edges_used++;
        }
    }

    if (edges_used != n - 1) {
        throw runtime_error("Graph is not connected");
    }
```

```
    return total_weight;
}
```

Complexity: `O(E log V)`.

24.8 Prim's Algorithm

Prim computes a minimum spanning tree by growing one tree and always adding the cheapest edge leaving the current tree.

```
long long prim(const Graph& graph) {
    int n = graph.size();

    vector<bool> in_mst(n, false);
    vector<long long> min_edge(n, INF);

    priority_queue<
        pair<long long, int>,
        vector<pair<long long, int>>,
        greater<pair<long long, int>>
    > pq;

    min_edge[0] = 0;
    pq.push({0, 0});

    long long total_weight = 0;
    int vertices_used = 0;

    while (!pq.empty()) {
        auto [weight, u] = pq.top();
        pq.pop();

        if (in_mst[u]) continue;

        in_mst[u] = true;
        total_weight += weight;
        vertices_used++;

        for (auto [v, w] : graph[u]) {
            if (!in_mst[v] && w < min_edge[v]) {
                min_edge[v] = w;
                pq.push({w, v});
            }
        }
    }

    if (vertices_used != n) {
        throw runtime_error("Graph is not connected");
    }

    return total_weight;
}
```

Complexity with binary heap: `O(E log V)`.

24.9 Johnson's Algorithm: Core Reweighting Idea

Johnson's algorithm is used for all-pairs shortest paths with negative edges but no negative cycles. The implementation below gives the essential structure.

```
vector<vector<long long>> johnson(int n, vector<Edge> edges) {
    int artificial_source = n;

    vector<Edge> extended_edges = edges;

    for (int v = 0; v < n; v++) {
        extended_edges.push_back({artificial_source, v, 0});
    }

    auto [h, has_negative_cycle] =
        bellman_ford(n + 1, extended_edges, artificial_source);

    if (has_negative_cycle) {
        throw runtime_error("Negative cycle detected");
    }

    Graph reweighted_graph(n);

    for (const Edge& e : edges) {
        long long new_weight = e.w + h[e.u] - h[e.v];
        reweighted_graph[e.u].push_back({e.v, new_weight});
    }

    vector<vector<long long>> dist(n, vector<long long>(n, INF));

    for (int s = 0; s < n; s++) {
        vector<long long> d = dijkstra(reweighted_graph, s);

        for (int v = 0; v < n; v++) {
            if (d[v] < INF) {
                dist[s][v] = d[v] - h[s] + h[v];
            }
        }
    }

    return dist;
}
```

Complexity with binary heap: approximately `O(VE log V)`. With Fibonacci heaps, the theoretical bound improves.

24.10 Iterative DFS Using a Stack

DFS can be implemented without recursion by using an explicit stack. This is useful when the graph is large and recursive DFS may cause stack overflow.

```
vector<int> dfs_iterative(const vector<vector<int>>& graph, int source) {
    int n = graph.size();

    vector<bool> visited(n, false);
```

```
    vector<int> order;

    stack<int> st;
    st.push(source);

    while (!st.empty()) {
        int u = st.top();
        st.pop();

        if (visited[u]) continue;

        visited[u] = true;
        order.push_back(u);

        for (int v : graph[u]) {
            if (!visited[v]) {
                st.push(v);
            }
        }
    }

    return order;
}
```

Complexity: `O(V + E)`.

25 Important String Algorithms in C++

25.1 Naive String Matching

The naive algorithm tries every possible starting position in the text and compares the pattern character by character.

```
#include <bits/stdc++.h>
using namespace std;

vector<int> naive_match(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();

    vector<int> occurrences;

    for (int i = 0; i + m <= n; i++) {
        bool ok = true;

        for (int j = 0; j < m; j++) {
            if (text[i + j] != pattern[j]) {
                ok = false;
                break;
            }
        }

        if (ok) {
            occurrences.push_back(i);
```

```
        }
    }

    return occurrences;
}
```

Complexity: $O(nm)$ in the worst case.

25.2 Polynomial Rolling Hash

Polynomial hashing allows us to compare substrings in constant time after linear preprocessing.

```
struct RollingHash {
    static const long long MOD = 1000000007;
    static const long long BASE = 911382323;

    vector<long long> h;
    vector<long long> power;

    RollingHash(const string& s) {
        int n = s.size();

        h.assign(n + 1, 0);
        power.assign(n + 1, 1);

        for (int i = 0; i < n; i++) {
            h[i + 1] = (h[i] * BASE + s[i]) % MOD;
            power[i + 1] = (power[i] * BASE) % MOD;
        }
    }

    long long get_hash(int l, int r) {
        // Returns hash of s[l ... r - 1]
        long long res = h[r] - (h[l] * power[r - l]) % MOD;

        if (res < 0) {
            res += MOD;
        }

        return res;
    }
};
```

Preprocessing: $O(n)$.
Substring hash query: $O(1)$.

25.3 Rabin-Karp String Matching

Rabin-Karp uses rolling hash to compare the pattern with every substring of the text of the same length.

```
vector<int> rabin_karp(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();
```

```
    vector<int> occurrences;

    if (m > n) {
        return occurrences;
    }

    RollingHash text_hash(text);
    RollingHash pattern_hash(pattern);

    long long target = pattern_hash.get_hash(0, m);

    for (int i = 0; i + m <= n; i++) {
        if (text_hash.get_hash(i, i + m) == target) {
            // Optional verification to avoid hash collision errors.
            if (text.substr(i, m) == pattern) {
                occurrences.push_back(i);
            }
        }
    }

    return occurrences;
}
```

Expected complexity: $O(n + m)$ if collisions are rare.
Worst case with many collisions: $O(nm)$ if we verify every collision.

25.4 Prefix Function for KMP

The prefix function $\pi[i]$ is the length of the longest proper prefix of `pattern[0 ... i]` that is also a suffix of `pattern[0 ... i]`.

```
vector<int> compute_pi(const string& pattern) {
    int m = pattern.size();

    vector<int> pi(m, 0);

    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];

        while (j > 0 && pattern[i] != pattern[j]) {
            j = pi[j - 1];
        }

        if (pattern[i] == pattern[j]) {
            j++;
        }

        pi[i] = j;
    }

    return pi;
}
```

Complexity: $O(m)$.

25.5 Knuth-Morris-Pratt Algorithm

KMP finds all occurrences of a pattern in a text in linear time.

```
vector<int> kmp_match(const string& text, const string& pattern) {
    int n = text.size();
    int m = pattern.size();

    vector<int> occurrences;

    if (m == 0) {
        return occurrences;
    }

    vector<int> pi = compute_pi(pattern);

    int j = 0;

    for (int i = 0; i < n; i++) {
        while (j > 0 && text[i] != pattern[j]) {
            j = pi[j - 1];
        }

        if (text[i] == pattern[j]) {
            j++;
        }

        if (j == m) {
            occurrences.push_back(i - m + 1);

            // Allows overlapping occurrences.
            j = pi[j - 1];
        }
    }

    return occurrences;
}
```

Complexity: $O(n + m)$.

25.6 Trie

A trie stores a set of strings by sharing common prefixes.

```
struct TrieNode {
    TrieNode* child[26];
    bool is_end;

    TrieNode() {
        for (int i = 0; i < 26; i++) {
            child[i] = nullptr;
        }
```

```
        is_end = false;
    }
};

struct Trie {
    TrieNode* root;

    Trie() {
        root = new TrieNode();
    }

    void insert(const string& s) {
        TrieNode* cur = root;

        for (char c : s) {
            int x = c - 'a';

            if (cur->child[x] == nullptr) {
                cur->child[x] = new TrieNode();
            }

            cur = cur->child[x];
        }

        cur->is_end = true;
    }

    bool contains(const string& s) {
        TrieNode* cur = root;

        for (char c : s) {
            int x = c - 'a';

            if (cur->child[x] == nullptr) {
                return false;
            }

            cur = cur->child[x];
        }

        return cur->is_end;
    }

    bool starts_with(const string& prefix) {
        TrieNode* cur = root;

        for (char c : prefix) {
            int x = c - 'a';

            if (cur->child[x] == nullptr) {
                return false;
            }

            cur = cur->child[x];
```

```
        }

        return true;
    }
};
```

Insertion/search complexity: $O(L)$, where L is the length of the word.

25.7 Suffix Array

A suffix array stores the starting positions of all suffixes of a string in lexicographic order.

```
vector<int> build_suffix_array(const string& s) {
    int n = s.size();

    vector<int> sa(n);
    vector<int> rank(n);
    vector<int> tmp(n);

    for (int i = 0; i < n; i++) {
        sa[i] = i;
        rank[i] = s[i];
    }

    for (int k = 1; k < n; k *= 2) {
        auto cmp = [&](int i, int j) {
            if (rank[i] != rank[j]) {
                return rank[i] < rank[j];
            }

            int ri = (i + k < n ? rank[i + k] : -1);
            int rj = (j + k < n ? rank[j + k] : -1);

            return ri < rj;
        };

        sort(sa.begin(), sa.end(), cmp);

        tmp[sa[0]] = 0;

        for (int i = 1; i < n; i++) {
            tmp[sa[i]] = tmp[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
        }

        for (int i = 0; i < n; i++) {
            rank[i] = tmp[i];
        }

        if (rank[sa[n - 1]] == n - 1) {
            break;
        }
    }

    return sa;
```

```
}
```

Complexity: $O(n \log^2 n)$ with comparison sorting.

25.8 Pattern Search with Suffix Array

Since the suffixes are sorted, we can binary search for a pattern.

```
bool starts_with_pattern(const string& s, int pos, const string& pattern)
   {
    int n = s.size();
    int m = pattern.size();

    if (pos + m > n) {
        return false;
    }

    for (int i = 0; i < m; i++) {
        if (s[pos + i] != pattern[i]) {
            return false;
        }
    }

    return true;
}

bool suffix_less_than_pattern(const string& s, int pos, const string&
   pattern) {
    int n = s.size();
    int m = pattern.size();

    int i = 0;

    while (pos + i < n && i < m) {
        if (s[pos + i] != pattern[i]) {
            return s[pos + i] < pattern[i];
        }

        i++;
    }

    return i == m ? false : true;
}

vector<int> suffix_array_match(
    const string& s,
    const vector<int>& sa,
    const string& pattern
) {
    int n = s.size();

    int left = 0;
    int right = n;
```

```
    while (left < right) {
        int mid = (left + right) / 2;

        if (suffix_less_than_pattern(s, sa[mid], pattern)) {
            left = mid + 1;
        } else {
            right = mid;
        }
    }

    vector<int> occurrences;

    for (int i = left; i < n; i++) {
        if (starts_with_pattern(s, sa[i], pattern)) {
            occurrences.push_back(sa[i]);
        } else {
            break;
        }
    }

    sort(occurrences.begin(), occurrences.end());

    return occurrences;
}
```

Basic query complexity: $O(m \log n + km)$, where k is the number of occurrences.

25.9 LCP Array: Kasai's Algorithm

The LCP array stores the longest common prefix between consecutive suffixes in the suffix array.

```
vector<int> build_lcp(const string& s, const vector<int>& sa) {
    int n = s.size();

    vector<int> rank(n);
    vector<int> lcp(n - 1);

    for (int i = 0; i < n; i++) {
        rank[sa[i]] = i;
    }

    int h = 0;

    for (int i = 0; i < n; i++) {
        int r = rank[i];

        if (r == n - 1) {
            h = 0;
            continue;
        }

        int j = sa[r + 1];

        while (i + h < n && j + h < n && s[i + h] == s[j + h]) {
```

```
            h++;
        }

        lcp[r] = h;

        if (h > 0) {
            h--;
        }
    }

    return lcp;
}
```

Complexity: $O(n)$ after the suffix array is built.

25.10 Longest Palindromic Subsequence

The longest palindromic subsequence of a string can be computed by dynamic programming.

```
int longest_palindromic_subsequence(const string& s) {
    int n = s.size();

    vector<vector<int>> dp(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        dp[i][i] = 1;
    }

    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len - 1;

            if (s[l] == s[r]) {
                dp[l][r] = 2;

                if (l + 1 <= r - 1) {
                    dp[l][r] += dp[l + 1][r - 1];
                }
            } else {
                dp[l][r] = max(dp[l + 1][r], dp[l][r - 1]);
            }
        }
    }

    return dp[0][n - 1];
}
```

Complexity: $O(n^2)$.

25.11 General Backtracking Framework

Backtracking is used when we build a solution step by step and abandon a partial solution as soon as it cannot lead to a valid complete solution.

The general structure is:

```
void backtrack(State& state) {
    if (is_complete(state)) {
        process_solution(state);
        return;
    }

    for (Choice choice : possible_choices(state)) {
        if (is_valid(choice, state)) {
            apply_choice(choice, state);

            backtrack(state);

            undo_choice(choice, state);
        }
    }
}
```

25.12 Backtracking Template in C++

```
#include <bits/stdc++.h>
using namespace std;

struct State {
    vector<int> solution;
};

bool is_complete(const State& state) {
    // Return true if the current partial solution is complete.
    return false;
}

vector<int> possible_choices(const State& state) {
    // Return the list of choices that we can try from this state.
    return {};
}

bool is_valid(int choice, const State& state) {
    // Return true if adding this choice keeps the state valid.
    return true;
}

void apply_choice(int choice, State& state) {
    // Add the choice to the current solution.
    state.solution.push_back(choice);
}

void undo_choice(int choice, State& state) {
    // Remove the choice and restore the previous state.
    state.solution.pop_back();
}

void process_solution(const State& state) {
```

```
    // Use or store the complete solution.
    for (int x : state.solution) {
        cout << x << " ";
    }
    cout << "\n";
}

void backtrack(State& state) {
    if (is_complete(state)) {
        process_solution(state);
        return;
    }

    for (int choice : possible_choices(state)) {
        if (is_valid(choice, state)) {
            apply_choice(choice, state);

            backtrack(state);

            undo_choice(choice, state);
        }
    }
}
```

25.13 Example 1: Generate All Subsets

Given an array, we decide for each element whether to take it or not.

```
#include <bits/stdc++.h>
using namespace std;

vector<int> a;
vector<int> current;
vector<vector<int>> all_subsets;

void backtrack_subsets(int i) {
    if (i == (int)a.size()) {
        all_subsets.push_back(current);
        return;
    }

    // Choice 1: do not take a[i]
    backtrack_subsets(i + 1);

    // Choice 2: take a[i]
    current.push_back(a[i]);
    backtrack_subsets(i + 1);
    current.pop_back();
}

int main() {
    a = {1, 2, 3};

    backtrack_subsets(0);
```

```
    for (auto subset : all_subsets) {
        cout << "{ ";
        for (int x : subset) {
            cout << x << " ";
        }
        cout << "}\n";
    }

    return 0;
}
```

Complexity: $O(2^n)$ subsets.

25.14 Example 2: Generate All Permutations

We build the permutation position by position. At each step, we choose an unused element.

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> perm;
vector<bool> used;

void backtrack_permutations() {
    if ((int)perm.size() == n) {
        for (int x : perm) {
            cout << x << " ";
        }
        cout << "\n";
        return;
    }

    for (int x = 1; x <= n; x++) {
        if (!used[x]) {
            used[x] = true;
            perm.push_back(x);

            backtrack_permutations();

            perm.pop_back();
            used[x] = false;
        }
    }
}

int main() {
    n = 3;
    used.assign(n + 1, false);

    backtrack_permutations();

    return 0;
```

```
}
```

Complexity: $O(n!)$ permutations.

25.15 Example 3: Generate All Combinations of Size k

We choose k elements among $\{1, \ldots, n\}$.

```
#include <bits/stdc++.h>
using namespace std;

int n, k;
vector<int> comb;

void backtrack_combinations(int start) {
    if ((int)comb.size() == k) {
        for (int x : comb) {
            cout << x << " ";
        }
        cout << "\n";
        return;
    }

    for (int x = start; x <= n; x++) {
        comb.push_back(x);

        backtrack_combinations(x + 1);

        comb.pop_back();
    }
}

int main() {
    n = 5;
    k = 3;

    backtrack_combinations(1);

    return 0;
}
```

Complexity: $O\binom{n}{k}$ generated solutions.

25.16 Example 4: N-Queens Backtracking

Place n queens on an $n \times n$ chessboard so that no two queens attack each other.

```
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> queen_col;

// used_col[c] tells whether column c already has a queen.
// used_diag1[r + c] corresponds to one diagonal direction.
```

```
// used_diag2[r - c + n - 1] corresponds to the other diagonal direction.
vector<bool> used_col;
vector<bool> used_diag1;
vector<bool> used_diag2;

void print_solution() {
    for (int r = 0; r < n; r++) {
        for (int c = 0; c < n; c++) {
            if (queen_col[r] == c) {
                cout << "Q";
            } else {
                cout << ".";
            }
        }
        cout << "\n";
    }
    cout << "\n";
}

void backtrack_queens(int row) {
    if (row == n) {
        print_solution();
        return;
    }

    for (int col = 0; col < n; col++) {
        int d1 = row + col;
        int d2 = row - col + n - 1;

        if (used_col[col] || used_diag1[d1] || used_diag2[d2]) {
            continue;
        }

        queen_col[row] = col;
        used_col[col] = true;
        used_diag1[d1] = true;
        used_diag2[d2] = true;

        backtrack_queens(row + 1);

        queen_col[row] = -1;
        used_col[col] = false;
        used_diag1[d1] = false;
        used_diag2[d2] = false;
    }
}

int main() {
    n = 8;

    queen_col.assign(n, -1);
    used_col.assign(n, false);
    used_diag1.assign(2 * n - 1, false);
    used_diag2.assign(2 * n - 1, false);
```

```
    backtrack_queens(0);

    return 0;
}
```